

Typklassen, Rekursion & Currying

Typklassen, Rekursion & Currying

Gliederung



- 1. Typklassen
- 2. Rekursion & Currying
- 3. Typklassen + Fehler in der Praxis
- 4. Übung

Typklassen

- Wir wollen **generisch programmieren**, aber nicht jeder Typ kann alles
- Beispiel: Eine generische `sort`-Funktion braucht **Vergleichbarkeit**
- OOP-Lösung: Interface implementieren
- Problem: Was, wenn wir den Typ gar nicht ändern können?

```
def sortInts(xs: List[Int]): List[Int] = xs.sorted
def sortStrings(xs: List[String]): List[String] = xs.sorted
```

```
// Wunsch:
// def sort[A](xs: List[A]): List[A] = ???
// Aber: Wie vergleichen wir beliebige A?
```

- Oft kommt der Typ aus einer **Bibliothek** oder aus Java
- Wir können ihn dann nicht einfach um eigene Interfaces erweitern
- Verhalten soll außerdem oft **außerhalb des Datentyps** definiert werden

```
import java.time.LocalDate
```

```
// Wie geben wir LocalDate aus,  
// ohne LocalDate selbst zu ändern?
```

- Statt den Typ zu ändern, definieren wir eine **externe Instanz**
- Das Verhalten lebt neben dem Typ, nicht in ihm

```
import java.time.LocalDate
```

```
trait Show[A] {  
  def show(a: A): String  
}
```

```
given Show[LocalDate] with {  
  def show(d: LocalDate): String = d.toString  
}
```

- Eine Typklasse beschreibt **Verhalten für einen Typ**
- In Scala ist das meist ein `trait` mit einem Typparameter
- Für jeden konkreten Typ gibt es dann eine **Instanz**

```
trait Show[A] {  
  def show(a: A): String  
}
```

Typklassen

Erste Instanzen

- Eine Instanz sagt: **So funktioniert das Verhalten für genau diesen Typ**
- Scala 3 nutzt dafür `given`

```
trait Show[A] {  
  def show(a: A): String  
}
```

```
given Show[Int] with {  
  def show(a: Int): String = a.toString  
}
```

```
given Show[Boolean] with {  
  def show(a: Boolean): String = if (a) "ja" else "nein"  
}
```


- Auch für eigene Datentypen definieren wir einfach eine Instanz
- Der Datentyp selbst muss dafür **nicht verändert** werden

```
case class User(name: String, age: Int)
```

```
given Show[User] with {  
  def show(u: User): String = s"${u.name} (${u.age} Jahre)"  
}
```

Typklassen

given und using

- `given` stellt eine Instanz bereit
- `using` fordert eine Instanz an
- Der Compiler sucht automatisch die passende Instanz

```
def display[A](a: A)(using s: Show[A]): String =  
    s.show(a)
```

```
display(42)                // Compiler setzt Show[Int] ein  
display(User("Alice", 25)) // Compiler setzt Show[User] ein
```

Typklassen

using explizit gedacht

- Der Aufruf wirkt kurz, weil der Compiler den `using`-Parameter ergänzt
- Gedanklich passiert aber ein ganz normaler Funktionsaufruf mit extra Argument

```
val user = User("Alice", 25)  
val userShow = summon[Show[User]]
```

```
display(user)  
display(user)(using userShow)
```

Typklassen

Scala 2: implicits

- In Scala 2 liefen Typklassen meist über das Keyword `implicit`
- Dasselbe Grundprinzip, aber mit **einem** sehr überladenen Mechanismus
- `implicit` wurde für Instanzen, Parameter, Conversions und Klassen benutzt

```
trait Show[A] {  
  def show(a: A): String  
}
```

```
implicit val userShow: Show[User] =  
  (u: User) => s"${u.name} (${u.age} Jahre)"
```

```
def display[A](a: A)(implicit s: Show[A]): String =  
  s.show(a)
```

Typklassen

implicit vs. given/using

- Scala 3 trennt klarer: `given` bereitstellen, `using` anfordern
- Das macht Typklassen **lesbarer** und die Rollen eindeutiger
- Fachlich ist es keine neue Idee, sondern eine bessere Syntax und Struktur

// Scala 2

```
implicit val userShow: Show[User] = ???  
def display[A](a: A)(implicit s: Show[A]): String = ???
```

// Scala 3

```
given Show[User] with { ... }  
def display[A](a: A)(using s: Show[A]): String = ???
```

Wo sucht der Compiler nach givens?

- Im aktuellen Scope
- In importierten Objekten
- Im Companion Object des Typs oder der Typklasse
- Es muss genau **eine passende** Instanz geben

- Context Bound ist Kurzschreibweise für einen `using`-Parameter
- Mit `summon` holen wir die Instanz explizit hervor

```
def display[A: Show](a: A): String =  
    summon[Show[A]].show(a)
```

```
def sort[A: Ordering](xs: List[A]): List[A] =  
    xs.sorted
```

- Typklassen werden oft mit **Extension Methods** kombiniert
- Dann wirkt das Verhalten wie eine eingebaute Methode

```
object ShowSyntax {  
  extension [A](a: A) {  
    def show(using s: Show[A]): String = s.show(a)  
  }  
}
```

```
import ShowSyntax.*
```

```
User("Alice", 25).show
```


- Givens sind **Kontext** und können lokal überschrieben werden
- So kann derselbe Typ in unterschiedlichen Kontexten unterschiedlich dargestellt werden

```
given Show[User] with {  
  def show(u: User): String = u.name  
}
```

```
def debug(user: User): String = {  
  given Show[User] with {  
    def show(u: User): String = s"User(name=${u.name}, age=${u.age})"  
  }  
  display(user)  
}
```

- Scala nutzt das Muster schon überall:
 - ``Ordering[A]``
 - ``Numeric[A]``
 - ``CanEqual[A, B]``

```
List(3, 1, 2).sorted
```

```
List("c", "a", "b").sorted
```

```
def sumAll[A](xs: List[A])(using n: Numeric[A]): A =  
  xs.foldLeft(n.zero)(n.plus)
```

Typklassen

Dictionary Passing

- Konzeptionell ist eine Typklasse ein **Wörterbuch von Funktionen**
- `using` heißt praktisch: Dieses Wörterbuch wird als Argument übergeben

```
def display[A](a: A)(using s: Show[A]): String =  
    s.show(a)
```

```
val userShow: Show[User] = summon[Show[User]]
```

```
def displayManual[A](a: A, s: Show[A]): String =  
    s.show(a)
```

```
displayManual(User("Alice", 25), userShow)
```

Rekursion & Currying

Rekursion & Currying

Currying



- Eine Funktion mit mehreren Parametern wird als Kette einstelliger Funktionen gedacht
- In Scala entspricht das mehreren Parameterlisten
- Das ist die direkte Verbindung zum Lambda-Kalkül

```
def add(a: Int)(b: Int): Int = a + b
```

```
add(2)(3) // → 5
```

Rekursion & Currying

Partielle Anwendung

- Gibt man nur die erste Parameterliste an, erhält man eine neue Funktion
- So entstehen spezialisierte Varianten sehr elegant

```
def multiply(factor: Int)(x: Int): Int = factor * x
```

```
val double = multiply(2)  
List(1, 2, 3).map(double)
```

- **Partielle Anwendung** und **Partial Function** sind zwei verschiedene Konzepte
- Partial Function: nur auf einem **Teil** der Eingaben definiert
- Ein `case`-Block mit Patterns ist oft genau eine `PartialFunction`

```
val pf: PartialFunction[Int, String] = {  
  case 0 => "null"  
  case n if n > 0 => "positiv"  
}
```

```
pf.isDefinedAt(5)
```

```
pf.isDefinedAt(-3)
```

Rekursion & Currying

Partial Function und collect

- `collect` nimmt eine `PartialFunction`
- Es kombiniert **Filtern** und **Transformieren** in einem Schritt

```
val xs = List(Some(1), None, Some(3))
```

```
xs.collect { case Some(v) => v * 10 }  
// → List(10, 30)
```


Rekursion & Currying

Partial Function komponieren



- Partial Functions lassen sich mit `orElse` zusammensetzen
- Mehrere kleine `case`-Blöcke lassen sich zu einer größeren Fallunterscheidung kombinieren

```
val negatives: PartialFunction[Int, String] = { case n if n < 0 => "negativ" }
val nonNegatives: PartialFunction[Int, String] = {
  case 0 => "null"
  case n if n > 0 => "positiv"
}
```

```
val classify = negatives.orElse(nonNegatives)
classify(-1)
```

Rekursion & Currying

Partial Function: Default-Fall

- Mit `applyOrElse` kann man einen Default-Fall ergänzen

```
val positives: PartialFunction[Int, String] = {  
  case n if n > 0 => "positiv"  
}
```

```
positives.applyOrElse(5, (_: Int) => "sonst")  
positives.applyOrElse(-2, (_: Int) => "sonst")
```

Rekursion & Currying

Currying und using

- `using` ist technisch nur eine weitere Parameterliste
- Typklassen passen deshalb sehr gut zu currierten Funktionen

```
def display[A](a: A)(using s: Show[A]): String =  
  s.show(a)
```

Rekursion & Currying

Rekursion: Grundidee

- Eine Funktion ruft sich selbst auf, um ein kleineres Teilproblem zu lösen
- Jede Rekursion braucht einen **Basisfall** und einen **rekursiven Fall**

```
def sumTo(currentValue: Long): Long = {  
    if (currentValue > 0) currentValue + sumTo(currentValue - 1)  
    else currentValue  
}
```

Rekursion & Currying

Das Stack-Problem

- Nicht-endrekursive Funktionen wachsen mit jedem Aufruf auf dem Stack
- Bei großen Eingaben droht ein `StackOverflowError`

sumTo(4)

→ 4 + sumTo(3)

→ 4 + (3 + sumTo(2))

→ 4 + (3 + (2 + sumTo(1)))

Rekursion & Currying

Endrekursion



- **Endrekursiv** heißt: der rekursive Aufruf ist die letzte Operation
- Dann kann der Compiler in eine Schleife optimieren

```
import scala.annotation.tailrec
```

```
@tailrec
```

```
def sumToTailRec(currentValue: Long, sum: Long): Long = {  
    if (currentValue > 0) sumToTailRec(currentValue - 1, sum + currentValue)  
    else sum  
}
```

Rekursion & Currying

Rekursion auf Listen

- Listen sind selbst rekursive Datenstrukturen: `head :: tail`

```
import scala.annotation.tailrec
```

```
@tailrec
```

```
def sum(xs: List[Int], acc: Int = 0): Int = xs match {  
  case Nil          => acc  
  case head :: tail => sum(tail, acc + head)  
}
```

Typklassen + Fehler in der Praxis

Typklassen + Fehler in der Praxis

Beispiel: JsonEncoder

- Eine typische Typklasse in Anwendungen: Werte in JSON kodieren

```
trait JsonEncoder[A] {  
  def encode(a: A): String  
}
```

Typklassen + Fehler in der Praxis

JsonEncoder: Primitive Instanzen

- Für primitive Typen definieren wir direkte Instanzen

```
given JsonEncoder[Int] with {  
  def encode(a: Int): String = a.toString  
}
```

```
given JsonEncoder[String] with {  
  def encode(a: String): String = s"\\"$a\""
```

```
given JsonEncoder[Boolean] with {  
  def encode(a: Boolean): String = a.toString  
}
```

Typklassen + Fehler in der Praxis

JsonEncoder: Eigene Typen

- Für eigene Typen kombinieren wir vorhandene Instanzen

```
given JsonEncoder[Double] with {  
  def encode(a: Double): String = a.toString  
}
```

Typklassen + Fehler in der Praxis

JsonEncoder: Student zusammensetzen

- Der Encoder für `Student` nutzt die Encoder für `String` und `Double`

```
case class Student(name: String, grade: Double)

given JsonEncoder[Student] with {
  def encode(s: Student): String = {
    val name = summon[JsonEncoder[String]].encode(s.name)
    val grade = summon[JsonEncoder[Double]].encode(s.grade)
    s"{\"name\": $name, \"grade\": $grade}"
  }
}
```

Typklassen + Fehler in der Praxis

Service-Pipeline mit Either



- Typisches Muster: Request prüfen, Daten laden, Ergebnis liefern

```
enum AppError {  
    case NotFound(id: Int)  
    case InvalidInput(msg: String)  
}
```

```
def parseId(raw: String): Either[AppError, Int] =  
    Either.catchOnly[NumberFormatException](raw.toInt)  
        .left.map(_ => AppError.InvalidInput("Ungültige Zahl"))
```

Typklassen + Fehler in der Praxis

Service-Pipeline: Verkettung



- `Either` macht den Happy Path linear und lesbar

```
case class User(id: Int, name: String)
```

```
def findUser(id: Int): Either[AppError, User] =  
  if (id == 1) Right(User(1, "Alice")) else Left(AppError.NotFound(id))
```

```
def loadUser(rawId: String): Either[AppError, User] =  
  parseId(rawId).flatMap(findUser)
```

Typklassen + Fehler in der Praxis

Fehler benutzerfreundlich rendern

- Typklassen und Fehlerbehandlung lassen sich direkt kombinieren

```
given Show[AppError] with {  
  def show(e: AppError): String = e match {  
    case AppError.NotFound(id)      => s"User $id nicht gefunden"  
    case AppError.InvalidInput(msg) => s"Ungültige Eingabe: $msg"  
  }  
}
```

Typklassen + Fehler in der Praxis

Best Practices



- **Typklassen** für Verhalten, das vom Typ abhängt
- **Endrekursion** bevorzugen, wenn Rekursion unvermeidlich ist
- **Currying** nutzen, wenn Parameter logisch in Gruppen gehören
- Fehler möglichst früh **typisieren**

Übung

- Implementiere die Typklasse ``JsonEncoder[A]``
- Starte mit Instanzen für ``Int``, ``String`` und ``Boolean``
- Tipp: orientiere dich an ``Show[A]`` aus dem Vorlesungsteil

- Ergänze ``JsonEncoder[Student]``
- Verwende die vorhandenen Encoder für die Felder
- Tipp: ``String`` und ``Double`` nicht direkt selbst formatieren, sondern vorhandene Encoder nutzen

```
case class Student(name: String, grade: Double)
```

- Implementiere generische Instanzen für `Option[A]` und `List[A]`
- Tipp: du brauchst einen vorhandenen Encoder für `A` als Kontextparameter

```
// Denk an:  
// given [A](using enc: JsonEncoder[A]) ...
```

- Ergänze eine `toJson`-Extension-Method
- Tipp: die Methode soll intern nur den passenden `JsonEncoder[A]` verwenden
- Siehe Vorlesungsfolie `Extension Methods für Ergonomie`

- Definiere ein lokales `given JsonEncoder[Student]`
- Tipp: lokal soll ein anderes Rendering aktiv sein als global

```
def debugJson(s: Student): String = {  
    given JsonEncoder[Student] // ??? expliziten JsonEncoder implementieren  
    ??? // Methode aufrufen, welche einen given JsonEncoder[Student] benötigt  
}
```

- Schreibe eine `PartialFunction[Double, String]` für Notenbereiche
- Tipp: sie muss nicht für alle Noten definiert sein

```
// Beispiel-Idee:  
// case g if ... => ...
```

- Extrahiere nur die bestandenenen Noten und transformiere sie
- Tipp: `collect` kombiniert Filtern und Transformieren in einem Schritt

```
val grades = List(1.3, 2.0, 4.0, 5.0)
```


- Implementiere eine endrekursive Listenfunktion
- Tipp: arbeite mit einem Akkumulator und `@tailrec`

```
import scala.annotation.tailrec
```

```
// z. B. reverse, length oder sum
```

- Schreibe eine curryierte Hilfsfunktion
- Tipp: wähle ein kleines Beispiel wie `multiply` oder `addPrefix`

```
// Zielidee:  
// def f(a)(b) = ...  
// val specialized = f(...)
```

- Ausgangsdaten für die Abschlusssaufgabe vorbereiten
- Tipp: überlege schon hier, welche Bausteine du gleich kombinieren willst

```
val students = List(  
    Student("Alice", 1.3),  
    Student("Bob", 4.0),  
    Student("Charlie", 5.0)  
)
```

- Kombiniere jetzt alles in einer kleinen Pipeline
- Tipp: kombiniere mindestens zwei der vorherigen Bausteine bewusst zusammen

```
// Mögliche Bausteine:  
// - JsonEncoder  
// - collect mit PartialFunction  
// - endrekursive Hilfsfunktion  
// - currying / partielle Anwendung
```